

VAN Pipeline

Code Organization & Function Reference

A stereo visual-SLAM pipeline with 7 sequential stages:

feature matching > stereo rectification > temporal tracking > tracking database

> bundle adjustment > pose-graph optimization > loop closure

Generated: July 15, 2026 at 16:51

1. Pipeline Architecture Overview

The code is organized around a sequential pipeline driven by `main.py`, which orchestrates seven exercise stages across `main.py` and six utility modules in `utils/`. Each stage ingests data from the previous stage, produces intermediate results (databases, poses, graphs), and feeds them forward. The TrackingDB data structure (`tracking_database.py`) is central to Exercises 4-7, persisting long-term feature tracks across frames.

`code/project/main.py`

Role: Top-level orchestration script for the entire VAN pipeline.

Purpose: Sequences all 7 exercise stages end-to-end (feature detection > stereo/triangulation > temporal PnP tracking > TrackingDB construction > GTSAM bundle adjustment > pose-graph optimization > loop-closure detection) and saves visualization plots to `outputs/`.

Function

`code/project/utils/geometry.py`

Role: Core low-level geometry, image I/O, camera calibration, feature extraction, triangulation, and single/multi-frame PnP-RANSAC

Purpose: Provides the foundational building blocks for feature matching (ORB/AKAZE), stereo rectification validation, 3D triangulation, and frame-to-frame visual-odometry (temporal matching + RANSAC pose estimation).

Function

`code/project/utils/tracking.py`

Role: Construction and analysis of the long-term multi-frame TrackingDB feature-track data structure.

Purpose: Implements sequential frame-by-frame PnP-RANSAC-driven track building (populating the TrackingDB), persistence (pickle load/save), and track statistics/analysis (reprojection error, triangulation).

Function

`code/project/utils/bundle_adjustment.py`

Role: GTSAM-based local bundle-adjustment utilities for sliding-window optimizations.

Purpose: Builds stereo-calibrated GTSAM factor graphs, constructs stereo reprojection factors, selects keyframes, and solves bundle-adjustment optimizations (feeding relative pose/covariance constraints to the pose graph).

Function

`code/project/utils/pose_graph.py`

Role: Global pose-graph construction, initialization, and optimization from relative pose/covariance constraints.

Purpose: Builds GTSAM pose-graph from bundle-adjustment edge constraints, optimizes the full keyframe trajectory, extracts relative poses/covariances via Schur-complement reduction, and supports prior-sensitivity analysis.

Function

`code/project/utils/loop_closure.py`

Role: Loop-closure detection, verification, and integration into the global pose graph.

Purpose: Detects candidates via Mahalanobis-distance filtering, verifies them with feature matching + fundamental-matrix RANSAC, estimates relative poses via small bundle adjustments, and adds loop-closure factors back to the pose graph.

Function

code/project/utils/visualization.py

Role: Centralized plotting and visualization library for the entire pipeline.

Purpose: Provides stereo-image displays, match/inlier overlays, 3D point clouds, trajectory plots, GTSAM bundle-adjustment/pose-graph visualizations (including covariance ellipsoids), and loop-closure comparison plots.

Function

code/tracking_database.py

Role: Core data structure for storing feature tracks across frames.

Purpose: Defines TrackingDB and Observation classes, which maintain track-frame mappings (track_id $\leftrightarrow$ frame_id), stereo observations, and serialization (pickle save/load).

Function

2. Key Pipeline Stages - Function/File/Line Reference

This section pinpoints the exact implementation location (file, function name, starting line number) for the nine core stages required for the oral exam. Each subsection includes a brief algorithmic description and all relevant function citations.

1. Triangulation

Converts matched 2D stereo correspondences into 3D world points using linear SVD (Direct Linear Transform) or OpenCV's homogeneous solution.

code/project/utils/geometry.py:131

> triangulate_point_linear

Triangulates one 3D point from a stereo pair via linear SVD.

code/project/utils/geometry.py:150

> triangulate_points_linear

Vectorized wrapper applying triangulate_point_linear over multiple pairs.

code/project/utils/geometry.py:155

> triangulate_points_opencv

Batch triangulation via cv2.triangulatePoints + homogeneous-to-Euclidean conversion.

code/project/utils/tracking.py:250

> triangulate_track_reference_point

Triangulates the 3D reference point for a track from its first stereo observation.

code/project/utils/bundle_adjustment.py:180

> build_and_solve_bundle_core

(call site) Triangulates each landmark's initial 3D estimate before GTSAM optimization.

2. RANSAC

Robust estimation of camera pose using RANSAC (Random Sampling Consensus). Samples minimal 4-point sets, solves PnP for each, and counts 4-camera reprojection supporters to find the best inlier hypothesis.

code/project/utils/geometry.py:184

> run_custom_pnp_ransac

Custom PnP-RANSAC with early stopping: samples 4 correspondences, solves PnP (EPNP), evaluates inliers via 4-camera reprojection.

code/project/utils/tracking.py:158

> build_data

(loop, inline) Same RANSAC logic inlined for persistent TrackingDB construction.

code/project/utils/loop_closure.py:163

> estimate_loop_relative_pose_bundle

(call site) Uses cv2.solvePnP Ransac for initial loop-closure pose guess.

3. PnP Trajectory Calculation

Estimates camera pose at each frame via Perspective-n-Point (PnP) solvers, composing relative poses across frames to build the full camera trajectory.

code/project/utils/geometry.py:282

```
> estimate_next_pose_with_rejection
```

One trajectory step: temporal matching > RANSAC pose > step-size/inlier rejection > pose composition.

code/project/main.py:250

```
> track_trajectory_over_sequence
```

Iterates frames, calling estimate_next_pose_with_rejection to accumulate global pose and build trajectory.

code/project/utils/tracking.py:127

```
> build_data
```

The 'real' trajectory used downstream: constructs TrackingDB while accumulating camera_poses via RANSAC + composition.

4. Database Definition

The TrackingDB data structure: stores feature tracks across frames with bidirectional mappings (track_id <-> frame_id, keyed observations in stereo pixel coords).

code/tracking_database.py:18

```
> class Observation
```

Dataclass: one stereo observation (frame_id, feature_idx, x_left, x_right, y).

code/tracking_database.py:26

```
> class TrackingDB
```

Core tracking database. __init__ (line 29) initializes maps: track_to_frames, frame_to_tracks, observations, feature_to_track.

5. Adding a Frame to the Database

Ingests a frame's temporal feature matches into TrackingDB, linking observations across time and either continuing existing tracks or creating new ones.

code/tracking_database.py:76

```
> TrackingDB.update_tracks
```

Main ingestion entry point: for each temporal match, either continues an existing track or creates a new one, inserting observations.

code/tracking_database.py:103

```
> TrackingDB.create_track
```

Allocates a new track ID and initializes its frame list.

code/tracking_database.py:110

```
> TrackingDB.add_observation
```

Low-level insert: builds Observation and updates all four internal maps.

code/project/utils/tracking.py:67

```
> add_feature_to_db
```

Helper called by update_tracks: inserts a single feature observation into the database.

6. Single Bundle Creation

Builds and solves a GTSAM factor graph over a sliding window of frames, optimizing camera poses and 3D landmarks jointly via Levenberg-Marquardt.

code/project/utils/bundle_adjustment.py:123

```
> build_and_solve_bundle_core
```

Core builder: creates NonlinearFactorGraph, anchors start frame, initializes poses, triangulates landmarks, adds Huber-robust stereo factors, solves with LM.

code/project/utils/bundle_adjustment.py:220

```
> solve_bundle_window
```

Standard wrapper around build_and_solve_bundle_core for extracting optimized geometry.

code/project/utils/bundle_adjustment.py:235

```
> solve_bundle_with_prior_sigma
```

Wrapper allowing custom prior noise sigma (used for sensitivity experiments).

7. Relative Transformation + Covariance Extraction

After bundle optimization, extracts the relative pose and conditional covariance between two keyframes via Schur-complement reduction and Adjoint map transformation.

code/project/utils/pose_graph.py:144

```
> compute_relative_pose_and_covariance
```

Solves a bundle window, computes marginals, applies Schur-complement formula + Adjoint map to get conditional covariance in relative-pose frame.

code/project/utils/loop_closure.py:126

```
> estimate_loop_relative_pose_bundle
```

Loop-closure analog: builds 2-pose GTSAM bundle, solves, and extracts relative pose/covariance (line 194-201) via jointMarginalInformation.

8. PoseGraph Building

Constructs the global GTSAM pose graph from per-bundle relative-pose/covariance constraints, initializes it, and optimizes to find the maximum-a-posteriori keyframe trajectory.

code/project/utils/pose_graph.py:17

```
> covariance_to_noise_model
```

Converts a 6×6 covariance matrix into a regularized GTSAM Gaussian noise model.

code/project/utils/pose_graph.py:26

```
> find_component_start_keyframes
```

Identifies the starting keyframe of each disconnected sub-graph component.

code/project/utils/pose_graph.py:38

```
> build_pose_graph
```

Builds NonlinearFactorGraph, adds strong priors at component starts, adds BetweenFactors for each relative-pose edge.

code/project/utils/pose_graph.py:72

```
> build_and_initialize_pose_graph
```

Calls `build_pose_graph` and chains relative poses to produce initial Values estimate.

code/project/utils/pose_graph.py:100

```
> optimize_pose_graph
```

Solves the graph with Levenberg-Marquardt and returns result + Marginals.

code/project/utils/pose_graph.py:165

```
> compute_all_relative_constraints
```

Iterates consecutive keyframe pairs, computing relative pose/covariance for each via `solve_bundle_window`.

code/project/utils/pose_graph.py:182

```
> clean_pose_graph_edges
```

Filters out edges with invalid/ill-conditioned covariances before graph construction.

9. Loop Closure Detection and Factor Creation

Detects loop-closure candidates via Mahalanobis-distance filtering over accumulated path covariances, verifies them visually (fundamental-matrix RANSAC), and adds loop-closure factors to the pose graph.

code/project/utils/loop_closure.py:38

```
> detect_loop_closure_candidates
```

Builds NetworkX shortest-path graph, finds candidates below Mahalanobis threshold (Lie-algebra error over path covariance).

code/project/utils/loop_closure.py:80

```
> verify_loop_closures_consensus
```

Visually verifies candidates: AKAZE matching + `cv2.findFundamentalMat` RANSAC, requires inlier ratio/count thresholds.

code/project/utils/loop_closure.py:126

```
> estimate_loop_relative_pose_bundle
```

Estimates loop-closure relative pose + covariance via PnP + 2-pose GTSAM bundle adjustment.

code/project/utils/loop_closure.py:223

```
> add_loop_closures_and_optimize
```

Adds verified loop constraints: builds `gtsam.BetweenFactorPose3` (line 239) for each measurement and adds to graph, re-optimizes after each.

3. Full Function Inventory (Appendix)

Complete listing of all functions and classes in each file, with starting line number and brief description.

code/project/main.py

Line	Function/Class	Description
37	<code>detect_and_visualize_orb_features</code>	Detects ORB keypoints on a stereo pair and displays them.
53	<code>inspect_feature_descriptors</code>	Prints raw ORB descriptor byte arrays.
61	<code>compute_raw_bruteforce_matches</code>	Computes raw Hamming brute-force matches between left/right descriptors.
70	<code>apply_lowe_ratio_test</code>	Filters matches with Lowe's ratio test.
100	<code>run_feature_detection_and_matching_pipeline</code>	Runs Exercise 1: detect > inspect > match > ratio test.
119	<code>analyze_rectified_stereo_deviations</code>	Computes and plots y-axis pixel deviation of stereo matches.
137	<code>filter_matches_by_rectified_pattern</code>	Splits matches into inliers/outliers by y-deviation threshold.
150	<code>triangulate_and_compare_methods</code>	Triangulates 3D points via linear SVD and OpenCV, compares them.
180	<code>run_stereo_rectification_pipeline</code>	Runs Exercise 2: deviation analysis > filtering > triangulation.
187	<code>generate_next_frame_point_cloud</code>	Builds stereo/triangulation data and plots 3D landmark cloud.
197	<code>match_temporal_features_left_images</code>	Matches ORB/AKAZE descriptors between consecutive left images.
207	<code>estimate_motion_with_four_point_pnp</code>	Estimates camera motion via minimal 4-point cv2.solvePnP.
225	<code>find_four_view_reprojection_supporters</code>	Finds temporal matches that reproject into all four cameras.
238	<code>estimate_motion_with_pnp_ransac</code>	Estimates relative pose with custom PnP-RANSAC.
250	<code>track_trajectory_over_sequence</code>	Tracks camera trajectory across the full frame sequence.
281		

	run_temporal_tracking_and_pnp_pipeline	Runs Exercise 3: temporal matching > PnP > RANSAC > trajectory.
302	build_tracking_database	Loads or builds the long-term TrackingDB.
310	report_tracking_statistics	Computes and prints summary tracking statistics.
317	visualize_sample_track_patches	Visualizes cropped image patches for a long feature track.
331	analyze_track_connectivity	Computes and plots frame-to-frame track connectivity.
341	analyze_ransac_inlier_percentage	Plots per-frame PnP-RANSAC inlier percentage.
351	plot_track_length_distribution	Plots histogram of track-length distribution.
359	analyze_track_reprojection_error	Triangulates reference point and plots reprojection error drift.
379	run_tracking_database_pipeline	Runs Exercise 4: build DB > stats > visualizations > error analysis.
396	analyze_single_track_error_gtsam	Computes GTSAM stereo reprojection and factor errors for one track.
446	run_first_bundle_window	Solves and visualizes first 10-frame bundle window.
484	run_full_sliding_bundle_adjustment	Runs full sliding-window bundle adjustment over keyframes.
526	run_bundle_adjustment_pipeline	Runs Exercise 5: single-track error > first window > full sliding BA.
537	extract_relative_pose_constraints	Extracts relative pose/covariance constraints between keyframes.
568	optimize_pose_graph_from_constraints	Cleans, builds, and optimizes the keyframe pose graph.
618	run_pose_graph_optimization_pipeline	Runs Exercise 6: extract constraints > optimize pose graph.
630	detect_loop_candidates	Detects loop-closure candidates via Mahalanobis-distance filtering.
649	verify_loop_closures	Visually verifies loop candidates using fundamental-matrix RANSAC.
666	estimate_loop_closure_poses	Estimates relative poses for verified loop closures.
677	update_pose_graph_with_closures	

		Adds loop-closure constraints to pose graph and re-optimizes.
694	report_loop_closure_results	Generates final comparison plots/metrics.
729	run_loop_closure_pipeline	Runs Exercise 7: detect > verify > estimate > update > report.
748	main	Top-level entry point: sequences all 7 exercises end-to-end.

code/project/utils/geometry.py

Line	Function/Class	Description
18	read_images	Reads grayscale left/right KITTI images for a given frame index.
28	get_num_frames	Returns total number of frames in sequence 00.
34	read_cameras	Loads camera calibration matrices (K, P1, P2) from calib.txt.
47	read_ground_truth_poses	Reads KITTI ground-truth 3x4 pose matrices as (R, t) tuples.
59	get_orb_features	Extracts ORB keypoints/descriptors from an image.
65	get_akaze_features	Extracts AKAZE keypoints/descriptors from an image.
71	camera_center	Computes 3D world camera center $C = -R^T t$.
76	compose_transform	Composes two relative rigid transforms ($T_{new} = T_2 * T_1$).
81	project_point	Projects one 3D point to 2D pixel coordinates.
90	project_points_vectorized	Vectorized batch projection of N 3D points.
101	compute_rectified_stereo_deviations	Computes absolute y-coordinate deviations between stereo keypoints.
111	split_matches_by_rectified_pattern	Splits matches into inliers/outliers by y-deviation threshold.
124	get_matched_points	Extracts paired 2D point arrays from a match list.
131	triangulate_point_linear	Triangulates one 3D point from stereo via linear SVD (DLT).
150	triangulate_points_linear	Vectorized wrapper over triangulate_point_linear.
155	triangulate_points_opencv	Batch triangulation using cv2.triangulatePoints.
162	median_3d_distance	

		Computes median Euclidean distance between corresponding 3D point sets.
167	crop_around_point	Crops a square image region centered on a 2D point.
179	valid_stereo_obs	Validates stereo observation has non-degenerate disparity.
184	run_custom_pnp_ransac	Custom PnP-RANSAC: samples 4 correspondences, solves PnP, counts 4-camera supporters.
241	find_four_view_supporters	Finds temporal matches that reproject into all four cameras.
282	estimate_next_pose_with_rejection	One trajectory step: temporal match > RANSAC > rejection > composition.
311	run_single_pair	Full single-frame pipeline: feature detection > matching > filtering > triangulation.
346	build_pnp_correspondences	Builds 3D(frame0)-2D(frame1, stereo) correspondence records.
373	evaluate_supporters	Vectorized evaluation of correspondence inlier status by 4-camera reprojection.
396	estimate_relative_pose_ransac	Alternate PnP-RANSAC pose estimator with early stopping.

code/project/utls/tracking.py

Line	Function/Class	Description
15	class CompatibleUnpickler	Custom Unpickler that remaps legacy module references for backward compatibility.
52	build_stereo_dict	Builds a dict mapping left-image keypoint index to stereo match.
57	get_feature_observation	Extracts left-x/right-x/y pixel coordinates for a given feature.
67	add_feature_to_db	Inserts one feature observation into the TrackingDB.
75	compute_tracking_statistics	Computes summary statistics (track counts, frame links) from database.
93	print_tracking_statistics	Prints computed tracking statistics to console.
104	select_track_by_min_length	Finds the longest track meeting a minimum-length criterion.
112	compute_connectivity	Computes number of shared tracks between consecutive frame pairs.
122	compute_track_lengths	Returns list of track lengths at/above a minimum threshold.
127	build_data	

		Builds TrackingDB from scratch: sequential frame-by-frame stereo + PnP-RANSAC matching.
212	load or build db	Loads cached TrackingDB pickle if available, else builds and saves new.
234	select long track	Randomly selects a track with at least a minimum number of observations.
242	prepare_stereo_projection_matrices	Constructs left/right stereo projection matrices for given extrinsics.
250	triangulate_track_reference_point	Triangulates 3D reference point for a track from first observation.
267	compute_track_reprojection_errors	Computes left/right reprojection errors across all frames of a track.

code/project/utils/bundle_adjustment.py

Line	Function/Class	Description
16	init_gtsam_stereo_calibration	Builds GTSAM Cal3_S2Stereo calibration from camera matrices.
24	get_c2w_pose	Converts world-to-camera (R, t) extrinsics to camera-to-world Pose3.
31	get_gtsam_camera_pose	Fetches ground-truth pose at a frame as GTSAM Pose3.
36	create_stereo_factor	Constructs GenericStereoFactor3D from observation, noise, keys.
42	compute_stereo_reprojection_error	Computes total pixel L2 stereo reprojection error.
52	compute_single_factor_error	Evaluates scalar GTSAM factor-graph cost of one stereo factor.
64	extract_optimized_geometry	Extracts optimized camera positions and 3-sigma-filtered landmarks.
87	w2c_to_local_gtsam_pose	Converts world-to-camera pose to window-local GTSAM Pose3.
94	pose_translation_np	Extracts translation vector from GTSAM Pose3 as NumPy array.
99	choose_keyframes	Selects keyframe indices by accumulated travel-distance and gap criteria.
123	build_and_solve_bundle_core	Core builder: creates GTSAM graph, adds factors, solves with LM.
220	solve_bundle_window	Standard wrapper extracting optimized geometry.
235	solve_bundle_with_prior_sigma	Wrapper allowing custom prior noise sigma.

code/project/utils/pose_graph.py

Line	Function/Class	Description
17	covariance_to_noise_model	Converts 6×6 covariance to regularized GTSAM Gaussian noise model.
26	find_component_start_keyframes	Identifies starting keyframe of each disconnected component.
38	build_pose_graph	Builds NonlinearFactorGraph with priors and BetweenFactors.
56	extract_trajectory_and_ids	Generic helper extracting frame IDs and 3D translations.
67	extract_pose_graph_positions	Thin wrapper around extract_trajectory_and_ids.
72	build_and_initialize_pose_graph	Calls build_pose_graph and chains relative poses for initialization.
100	optimize_pose_graph	Solves graph with Levenberg-Marquardt, returns result + Marginals.
107	run_prior_sensitivity_sweep	Solves bundle under several prior noise scales, plots covariances.
144	compute_relative_pose_and_covariance	Computes relative pose/covariance via Schur-complement + Adjoint.
165	compute_all_relative_constraints	Iterates keyframe pairs computing relative pose/covariance.
182	clean_pose_graph_edges	Filters edges with invalid/ill-conditioned covariances.

code/project/utils/loop_closure.py

Line	Function/Class	Description
17	_shortest_path_pose_graph	Builds NetworkX graph weighted by covariance trace.
25	_get_edge_pose_and_cov	Retrieves relative pose/covariance between keyframe nodes.
38	detect_loop_closure_candidates	Finds candidates below Mahalanobis threshold via Dijkstra paths.
80	verify_loop_closures_consensus	Verifies candidates: AKAZE + fundamental-matrix RANSAC.
126	estimate_loop_relative_pose_bundle	Estimates loop relative pose + covariance via 2-pose GTSAM bundle.
210	estimate_verified_loop_relative_poses	Runs estimate_loop_relative_pose_bundle over verified pairs.

223	add_loop_closures_and_optimize	Adds loop-closure BetweenFactorPose3 constraints and re-optimizes.
-----	--------------------------------	--

code/project/utils/visualization.py

Line	Function/Class	Description
20	plot_stereo_side_by_side	Displays left/right stereo images side-by-side.
35	draw_matches_custom	Draws random subset of feature matches.
52	plot_deviation_histogram	Plots histogram of stereo y-deviation values.
62	print_large_deviation_percentage	Prints % of matches exceeding deviation threshold.
71	draw_inliers_outliers	Overlays inlier/outlier keypoint markers on stereo pair.
91	plot_3d_points	Plots 3D scatter of landmark point cloud.
104	plot_four_cameras	Plots bird's-eye-view positions of 4 stereo cameras at two times.
132	draw_match_canvas	Generic helper drawing green/red match lines.
160	draw_temporal_supporters	Draws temporal-tracking supporter/non-supporter matches.
165	draw_ransac_results	Visualizes PnP-RANSAC inlier/outlier temporal matches.
174	plot_transformed_clouds	Visualizes alignment of frame0 cloud transformed into frame1.
196	plot_trajectory	Plots estimated vs ground-truth trajectory (top-down X-Z view).
210	plot_track_observations	Displays crop-window image patches of track across frames.
238	plot_connectivity	Plots frame-to-frame track connectivity over time.
251	plot_inlier_percentage	Plots per-frame PnP-RANSAC inlier percentage.
265	plot_track_length_histogram	Plots log-scale histogram of track lengths.
279	plot_track_reprojection_error_analysis	Plots left/right reprojection errors across track's frames.
295	draw_projection_validation_frames	Draws measured vs projected points, saves to disk.
318	reprojection_error_graph	

		Plots and saves stereo reprojection errors across frame indices.
332	factor_error_graph	Plots and saves GTSAM factor-cost evolution.
344	bundle1_3D	Renders and saves 3D local bundle trajectory with camera axes.
374	marginal_covariances	Plots and saves 3D trajectory with marginal covariance ellipsoids.
398	bundle1_2D_full	Plots and saves top-down (X-Z) view of bundle window.
422	bundle1_2D_zoomed	Plots and saves zoomed 2D top-down trajectory comparison.
438	plot_q5_4_results	Plots and saves optimized sliding-window keyframe trajectory.
466	plot_keyframe_localization_error	Plots and saves keyframe localization error vs ground truth.
497	plot_pose_graph_trajectory	Plots and saves top-down 2D pose-graph trajectory.
518	plot_pose_graph_with_covariances	Plots and saves 3D pose-graph with covariance ellipsoid wireframes.
575	plot_pose_graph_2d_ellipses	Plots and saves top-down 2D trajectory with confidence ellipses.
625	plot_loop_candidates	Plots and saves trajectory with loop-closure candidate connections.
664	plot_pose_graph_vs_ground_truth	Plots and saves trajectories with/without loop closures vs GT.
703	plot_absolute_location_error	Plots and saves keyframe position error, with/without loop closures.
740	plot_location_uncertainty_size	Plots and saves marginal-covariance-derived positional uncertainty.

code/tracking_database.py

Line	Function/Class	Description
18	class Observation	Dataclass: one stereo observation (frame_id, feature_idx, x_left, x_right, y).
26	class TrackingDB	Core tracking database storing feature tracks across multiple frames.
29	TrackingDB.__init__	Initialize empty DB with maps: track_to_frames, frame_to_tracks, observations, feature_to_track.
52	TrackingDB.tracks	Returns list of track IDs visible in a given frame.
56	TrackingDB.frames	Returns list of frame IDs where a track appears.

60	TrackingDB.observation	Returns Observation for a feature in a frame and track.
64	TrackingDB.track_num	Returns total number of tracks.
68	TrackingDB.frame_num	Returns total number of frames.
76	TrackingDB.update_tracks	Main ingestion entry point: links temporal matches into tracks.
103	TrackingDB.create_track	Allocates new track ID and initializes its frame list.
110	TrackingDB.add_observation	Low-level insert: adds stereo observation and updates all maps.
127	TrackingDB.has_feature	Checks if a feature has been tracked in a given frame.
131	TrackingDB.get_track_of_feature	Returns track ID for a feature in a frame.
139	TrackingDB.save	Serializes database to pickle file.
144	TrackingDB.load	Deserializes database from pickle file (static method).